

GUÍA – PRIMERA REVIEW TÉCNICA DEL PROYECTO

OBJETIVO DE LA REVIEW

La primera review técnica tiene como propósito validar que el sistema desarrollado cumple con los siguientes aspectos:

- Funcionamiento end-to-end (Frontend + Backend + Base de Datos)
- Coherencia entre los componentes del sistema
- Implementación inicial de una arquitectura organizada
- Comprensión funcional de las primeras características desarrolladas

Esta revisión simula un entorno real de empresa, donde no solo se evalúa si el sistema funciona, sino cómo está construido y cómo puede evolucionar.

1. PRESENTACIÓN DEL EQUIPO

Duración estimada: 10 a 15 minutos por equipo

1.1 Portada

Debe incluir:

- Nombre del proyecto
- Integrantes del equipo
- Fecha

1.2 Problema que resuelve el sistema

El equipo debe explicar claramente:

- ¿Qué problema están solucionando?
- ¿Quién es el usuario objetivo?

Ejemplo:

“Simplificar la gestión de pedidos para pequeños comercios”

1.3 Alcance de la primera entrega

Se debe indicar:

- Funcionalidades que ya están implementadas
- Funcionalidades que aún no están desarrolladas

Ejemplo:

Incluye:

- Crear registros
- Consultar información

No incluye:

- Reportes
- Integraciones externas

1.4 Arquitectura general del sistema

Explicar de manera sencilla:

- Frontend (Angular)
- Backend (Spring Boot)
- Base de datos

Se debe describir cómo interactúan entre sí.

1.5 Diagrama de arquitectura

Debe presentarse un diagrama donde se evidencie:

- Usuario
- Frontend
- Backend
- Base de datos

Se debe explicar el flujo completo:

Usuario → Frontend → API → Base de Datos

1.6 Backend

El equipo debe mostrar:

- Estructura de capas (Controller, Service/UseCase, Repository)
- Flujo de una petición dentro del backend

1.7 Frontend

El equipo debe mostrar:

- Componentes principales
- Servicios de consumo de API

Se debe explicar cómo el frontend se comunica con el backend.

1.8 Demo técnica (Obligatoria)

Este es el punto más importante de la presentación.

DEMOSTRACIÓN DEL SISTEMA

2.1 Prueba de APIs con Postman

Se debe demostrar:

- Creación de registros (POST)
- Consulta de datos (GET)
- Actualización o eliminación (PUT/DELETE, si aplica)

Se debe explicar:

- Request
- Response
- Código de estado HTTP

2.2 Flujo completo (Frontend + Backend)

Se debe evidenciar:

1. Acción realizada en el frontend
2. Llamado al backend
3. Respuesta mostrada al usuario

2.3 Validación en Base de Datos

Se debe demostrar que:

- Los datos realmente se guardan
- Existe coherencia entre lo que se envía y lo que se almacena

Esta validación se realiza consultando directamente la base de datos.

1.9 Problemas encontrados

El equipo debe mencionar:

- Dificultades técnicas
- Errores encontrados
- Limitaciones actuales

1.10 Próximos pasos

Se debe indicar:

- Qué mejoras se planean
- Qué funcionalidades se desarrollarán
- Qué aspectos de arquitectura se deben ajustar

2. DOCUMENTO – VERSIÓN 2

Este documento representa un nivel más detallado del sistema.

2.1 Casos de uso

Ejemplo:

Caso de uso: Crear pedido

- Actor: Usuario
- Flujo:
 1. Ingresa datos
 2. Envía formulario
 3. El sistema guarda la información

2.2 Historias de Usuario (HU)

Formato:

“Como [tipo de usuario]
quiero [funcionalidad]
para [beneficio]”

Ejemplo:

“Como cliente
quiero registrar un pedido
para poder comprar productos”

2.3 Criterios de aceptación

Se deben definir condiciones claras para validar cada funcionalidad.

Ejemplo:

- El sistema debe guardar correctamente el pedido
- Los campos obligatorios deben validarse
- El sistema debe retornar una respuesta exitosa

3. DIAGRAMA DE ARQUITECTURA

3.1 Elementos obligatorios

El diagrama debe incluir:

- Usuario
- Frontend
- Backend
- Base de datos

3.2 Flujo del sistema

Se debe representar claramente:

Usuario → Frontend → Backend → Base de Datos

3.3 Consideraciones

- No es solo dibujar, se debe explicar
- Debe ser claro y entendible
- Debe reflejar lo que realmente está implementado

4. CRITERIOS DE VALIDACIÓN EN LA REVIEW

4.1 APIs

- Funcionan correctamente
- Responden con códigos HTTP adecuados
- Manejan errores

4.2 Flujo completo

- Frontend conectado al backend
- Backend procesando correctamente
- Respuestas visibles en la interfaz

4.3 Base de datos

- Persistencia real de datos
- Coherencia de la información

4.4 Arquitectura

- Separación de responsabilidades
- Organización del proyecto
- Buenas prácticas básicas

ERRORES CRÍTICOS (PENALIZACIONES)

- APIs que no funcionan
- Frontend sin conexión al backend
- Datos que no se guardan en la base de datos
- Código desorganizado
- Todo implementado en una sola capa
- Falta de comprensión del sistema